Normalization (1NF, 2NF, 3NF)

Agenda

- **Introduction to Normalization**
- **First Normal Form (1NF)**
- **Second Normal Form (2NF)**
- **Third Normal Form (3NF)**

Introduction to Normalization

Normalization

- **Normalization is an important process in database design that helps improve the database's efficiency, consistency, and accuracy.**
- **It makes it easier to manage and maintain the data and ensures that the database is adaptable to changing business needs.**

Why Normalization?

- **The primary objective for normalizing the relations is to eliminate the below anomalies:**
  - **Insertion Anomalies**
  - **Deletion anomalies**
  - **Updation anomalies**
- **Failure to reduce anomalies results in data redundancy, which may threaten data integrity and cause additional issues as the database increases.**

Why Normalization?

- **Insertion anomalies:** Insertion anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete.
- **Deletion anomalies:** Deletion anomalies occur when deleting a record from a database and can result in the unintentional loss of data.
- **Updation anomalies:** occur when modifying data in a database and can result in inconsistencies or errors.

Normalization

- **Database normalization is the process of organizing the attributes of the database to reduce or eliminate data redundancy (having the same data but at different places).**
- **Data redundancy unnecessarily increases the size of the database as the same data is repeated in many places. Inconsistency problems also arise during insert, delete, and update operations.**

Normal Forms

- **In the relational model, there exist standard methods to quantify how efficient a databases is. These methods are called normal forms and there are algorithms to covert a given database into normal forms.**
- **The different types of Normal Forms are:**
  - **First Normal Form (1NF)**
  - **Second Normal Form (2NF) (includes 1NF)**
  - **Third Normal Form (3NF) (includes 2NF + 1NF)**

Example

Before Normalization

Employee_Department

Emp_ID	Emp_Name	Department	Dept_Location	Emp_Skills
101	Nick Wise	HR	London	Recruitment,Payroll
102	John Cader	Finance	Australia	Budgeting
103	Lily Case	HR	London	Recruitment
104	Ford Dawid	IT	Chicago	Programming, Testing

Example

Problems in the Employee_Department Relation

1. **Insertion Anomaly:**
   - If a new department is created but no employee is assigned to it yet, we cannot store its location because we need an employee record to insert.
2. **Update Anomaly:**
   - If the location of the HR department changes, we must update it in multiple rows (for both Nick Wise and Lily Case). If one row is missed, the data becomes inconsistent.
3. **Deletion Anomaly:**
   - If all employees in the IT department leave, we lose the department information, including its location.
4. **Data Redundancy:**
   - The department location is repeated for every employee in the same department.

Example

After Normalization

Employee

Emp_ID	Emp_Name	Dept_ID
101	Nick Wise	D1
102	John Cader	D2
103	Lily Case	D1
104	Ford Dawid	D3

Department

Dept_ID	Department	Dept_Location
D1	HR	London
D2	Finance	Australia
D3	IT	Chicago

Employee_Skills

Emp_ID	Emp_Skills
101	Recruitment
101	Payroll
102	Budgeting
103	Recruitment
104	Programming
104	Testing

Example

- **Before Normalization: The table is prone to redundancy and anomalies (insertion, update, and deletion).**
- **After Normalization: The data is divided into logical tables to ensure consistency, avoid redundancy and remove anomalies making the database efficient and reliable.**

First Normal Form (1NF)

1NF

- **First Normal Form (1NF) ensures that the structure of a database table is organized in a way that makes it easier to manage and query.**
- **A relation is in first normal form if every attribute in that relation is single-valued attribute or it does not contain any composite or multi-valued attribute.**
- **It is the first and essential step in to reduce redundancy, improve data integrity and reducing anomalies in relational database design.**

1NF

- **A relation (table) is said to be in First Normal Form (1NF) if:**
  - **All the attributes (columns) contain only atomic (indivisible) values.**
  - **Each column contains values of a single type.**
  - **Each record (row) is unique, meaning it can be identified by a primary key.**
  - **There are no repeating groups or arrays in any row.**

1NF

- **The basic rules are as follows:**
  - **For composite attributes, each component becomes a separate attribute in the resultant 1NF**
  - **For multivalued attributes, each item becomes a separate tuple, however, when defining the 1NF schema, the primary keys must be updated**

1NF - Example

Example 1: Customer orders table

Imagine a **Customer Orders** table that looks like this:

CustomerID	Name	Product1	Product2	OrderDate
1	Alice	Apples	Bananas	2025-01-10
2	Bob	Oranges	Grapes	2025-01-11

What's wrong?

1NF - Example

What's wrong?

This table violates 1NF because it contains multiple columns, **Product1** and **Product2** that refer to the same type of data (products)l.

So how do we fix it?

`To achieve 1NF, split the data into separate rows so that each cell contains only one value:

1NF - Example

CustomerID	Name	Product	OrderDate
1	Alice	Apples	2025-01-10
1	Alice	Bananas	2025-01-10
2	Bob	Oranges	2025-01-11
2	Bob	Grapes	2025-01-11

1NF - Example

Before Normalization

StudentID	Name	Grades
101	Sarah	A, B, C
102	Michael	B, A, A

1NF - Example

After Normalization

StudentID	Name	Grade
101	Sarah	A
101	Sarah	B
101	Sarah	C
102	Michael	B
102	Michael	A
102	Michael	A

1NF - Conclusion

- **It is important to note that 1NF is not an end in itself as it doesn’t tackle all forms of redundancy.**
- **while 1NF is a solid foundation, it’s just the first step in database normalization and doesn’t fully optimize a database structure on its own.**

Second Normal Form (2NF)

Prime Attributes

- **An attribute of a relation schema, R that is a member of some candidate key of R, is called a prime attribute**
- **For example, given relation is:**
- **R (A, B, C, D, E, F)**
- **And the candidate keys are: {A, C}, {B, D}**
- **Therefore, prime attributes will be: A, B, C, D**

Non Prime Attributes

- **An attribute that is not prime**
- **Or an attribute of a relation schema, R that is not a member of some candidate key of R, is called a non-prime attribute**
- **For example, given relation is:**
- **R (A, B, C, D, E, F)**
- **And the candidate keys are: {A, C}, {B, D}**
- **Therefore, prime attributes will be: A, B, C, D**
- **And non-prime attributes will be: E, F**

Functional Dependency

- **Functional dependencies help identify redundancy in data.**
- **They are the basis for splitting tables into smaller, well-structured relations.**

Functional Dependency

- **A functional dependency X → Y exists if, for any two tuples (rows) in a relation, whenever the values of attributes in X are the same, then the values of attributes in Y must also be the same.**
- **X is called the determinant (the attribute(s) on the left side).**
- **Y is the dependent attribute(s) (the attribute(s) on the right side).**

Functional Dependency

- **Theorem:**
  - **A functional dependency denoted by X  Y between two sets of attributes, X and Y that are subsets of relation schema, R holds if and only if:**
    - **For any two tuples, t1 and t2, we have t1[X] = t2[X]**
    - **Then we must have t1[Y] = t2[Y]**

Functional Dependency Example

- **Identify functional dependencies in the following relation:**

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

Functional Dependency Example

- **Identify functional dependencies in the following relation:**
  - B → C
  - C → B
  - {A, B} → C
  - {A, B} → D
  - {C, D} → B

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

Example

- Consider the following relation →
- The attributes *building* and *budget* are functionally dependent on *dept_name*.
- Are there any other functional dependencies?

ID	*name*	*salary*	*dept_name*	*building*	*budget*
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

Figure 7.2 The *in_dep* relation.

Full Functional Dependency

- **Full Functional Dependency:**
  - **An FD: $X \rightarrow Y$ is a full functional dependency if removal of any attribute A from X means that the dependency does not hold anymore**
  - **That is, all attributes of attribute set, X must be used together to define functional dependency**

Full Functional Dependency

- **For example, consider the following schema: Emp_Proj (SSN, ProjectID, EmpName, WorkedHours, ProjectName, ProjectLocation)**
- **Check if the following is fully or partially dependent?**
  - **{SSN, ProjectID} -> WorkedHours?**
    - **For combined attributes, check if any subset will fulfill the dependency requirement**
    - **SSN -> WorkedHours? Not possible, as one employee can work on multiple projects so cannot tell hours he worked on a specific project just by SSN**
    - **ProjectID -> WorkedHours? Not possible, because many people work on a project, so if these are the number of hours an employee has worked on a project, then just the Project ID would be insufficient to deduce hours**
    - **Hence FULL FD**

Partial Functional Dependency

- **An FD: X -> Y is a partial functional dependency if after removing some attributes from the attribute set X, the functional dependency still holds.**
- **A partial dependency occurs when a non-prime attribute (column that isn't part of any candidate key) relies on only part of a composite key instead of the whole key.**
- *A composite key is a primary key that consists of two or more attributes (columns) used together to uniquely identify a row (tuple) in a table, when a single attribute alone is not sufficient.*

Partial Functional Dependency

- **For example, consider the following schema: Emp_Proj (SSN, ProjectID, EmpName, WorkedHours, ProjectName, ProjectLocation)**
- **Check if the following is fully or partially dependent?**
  - **{SSN, ProjectID} → EmpName?**
    - **For combined attributes, check if any subset will fulfill the dependency requirement**
    - **SSN → EmpName? Possible, as you can find an employee's name through their SSN**
    - **Hence PARTIAL FD**

Partial Functional Dependency

- **For example, consider the following schema:**
  Emp_Proj (SSN, ProjectID, EmpName, WorkedHours, ProjectName, ProjectLocation)
- **Check if the following is fully or partially dependent?**
  - **{SSN, ProjectID} → ProjectName?**
    - **For combined attributes, check if any subset will fulfill the dependency requirement**
    - **SSN → ProjectName? Not possible, as an employee can work on several projects so just by their identity we cannot uniquely find the name of the project they worked on**
    - **ProjectID → ProjectName? Possible, because you can find the name of the project through its ID**
    - **Hence PARTIAL FD**

Second Normal Form (2NF)

- **Before a table can be in 2NF, it must satisfy the rules of first normal form:**
  - **Atomicity: Each cell must contain a single value (no repeating groups or arrays).**
  - **Unique rows: The table must have a clear primary key.**
- **2NF goes one step further with an additional rule: eliminate partial dependencies.**

Second Normal Form (2NF)

- **Leaving partial dependencies in a table means that redundant data can creep into the database, leading to inefficiency and potential inconsistencies during updates or deletions.**
- **If the primary key contains a single attribute, the test need not be applied at all, as that is considered as full dependency by default**

Example

Student ID	Course ID	Course Name	Instructor Name
1001	201	SQL Fundamentals	Ken Smith
1002	202	Introduction to Python	Merlin O'Donnell
1001	202	Introduction to Python	Merlin O'Donnell

1. Identify our candidate key.
In this case, the candidate key is a composite key of **Student ID** and **Course ID**. This unique combination identifies each row in the table.

Example

Student ID	Course ID	Course Name	Instructor Name
1001	201	SQL Fundamentals	Ken Smith
1002	202	Introduction to Python	Merlin O'Donnell
1001	202	Introduction to Python	Merlin O'Donnell

2. Determine our functional dependencies
Course Name and **Instructor Name** depend on **Course ID**, not the full composite key (**Student ID**, **Course ID**). This is a partial dependency because these attributes depend on only part of the composite key.

Example

Student ID	Course ID	Course Name	Instructor Name
1001	201	SQL Fundamentals	Ken Smith
1002	202	Introduction to Python	Merlin O'Donnell
1001	202	Introduction to Python	Merlin O'Donnell

3. Eliminate partial dependencies
We need to move the attributes that depend on only part of the key (**Course Name** and **Instructor Name**) to a new table that is based solely on **Course ID**.

Example

After decomposition, our new tables look like this:
Course enrollment table

Student ID	Course ID
1001	201
1002	202
1001	202

Course details table

Course ID	Course Name	Instructor Name
201	SQL Fundamentals	Ken Smith
202	Introduction to Python	Merlin O'Donnell

Example 2

Order ID	Product ID	Order Date	Product Name	Supplier Name
1	201	2024-11-01	Laptop	TechSupply
1	202	2024-11-01	Mouse	TechSupply
2	201	2024-11-02	Laptop	TechSupply
3	203	2024-11-03	Keyboard	KeyMasters

Third Normal Form (3NF)

3NF

- **A relation is in Third Normal Form (3NF) if it satisfies the following two conditions:**
  - **It is in Second Normal Form (2NF): This means the table has no partial dependencies (i.e., no non-prime attribute is dependent on a part of a candidate key).**
  - **There is no transitive dependency for non-prime attributes.**

Transitive Dependency

- **A functional dependency X → Y in a relation schema R is a transitive dependency**
  - **If there exists a set of attributes Z in R that is neither a candidate key nor a subset of any key of R, and**
  - **both X → Z and Z → Y hold**

Transitive Dependency

- **Simple words**
- **A transitive dependency occurs when one non-prime attribute depends on another non-prime attribute rather than depending directly on the primary key. This can create redundancy and inconsistencies in the database.**

Transitive Dependency

- **For example, if we have the following relationship between attributes:**
  - **A -> B (A determines B)**
  - **B -> C (B determines C)**
  - **This means that A indirectly determines C through B, creating a transitive dependency. 3NF eliminates these transitive dependencies to ensure that non-key attributes are directly dependent only on the primary key.**

3NF

- **Theorem:**
  - **For every Functional Dependency given in the schema, check X -> A**
  - **If one of the following conditions are satisfied, then the relation schema is considered to be in 3NF**
    - **X is a superkey of R**
    - **A is a prime attribute of R**

3NF - Example

- **Consider the following relation for a Candidate table with the following attributes and functional dependencies:**

CAND_NO	CAND_NAME	CAND_STATE	CAND_COUNTRY	CAND_AGE
1	Amayra	Sindh	Pakistan	18
2	Rihaan	Punjab	Pakistan	17
3	Manya	Khyber Pakhtunkhwa	Pakistan	19

3NF - Example

- **1. Functional dependency Set:**
  - **The set of functional dependencies is as follows:**
    - **CAND_NO → CAND_NAME**
    - **CAND_NO → CAND_STATE**
    - **CAND_STATE → CAND_COUNTRY**
    - **CAND_NO → CAND_AGE**

3NF - Example

- **2. Determining the Candidate Key:**
  - **The candidate key for this relation is {CAND_NO}, since CAND_NO uniquely identifies all other attributes in the table.**

3NF - Example

- **3. Identifying Transitive Dependency:**
  - **The issue here arises from the transitive dependency between CAND_NO and CAND_COUNTRY:**
- **CAND_NO → CAND_STATE**
- **CAND_STATE → CAND_COUNTRY**
- **This means that CAND_COUNTRY is transitively dependent on CAND_NO via CAND_STATE, which violates the Third Normal Form (3NF) rule**

3NF - Example

- **Converting the Relation into 3NF**
  - **To remove the transitive dependency and ensure the relation is in 3NF, we decompose the original CANDIDATE relation into two separate relations:**
    - **CANDIDATE (CAND_NO, CAND_NAME, CAND_STATE, CAND_AGE)**
    - **STATE_COUNTRY (CAND_STATE, CAND_COUNTRY)**

3NF - Example

CAND_NO	CAND_NAME	CAND_STATE	CAND_AGE
1	Amayra	Sindh	18
2	Rihaan	Punjab	17
3	Manya	Khyber Pakhtunkhwa	19

CAND_STATE	CAND_COUNTRY
Sindh	Pakistan
Punjab	Pakistan
Khyber Pakhtunkhwa	Pakistan

3NF - Exercise

- **For the given relation schema:**
  Emp_Dept (SSN, EmpName, Bdate, Address, DeptNum, DeptName, DeptManagerSSN)
  - **Birth date and address need not be divided into sub-parts**
  - **The possible functional dependencies are:**
  - **SSN → EmpName**
  - **SSN → Bdate**
  - **SSN → Address**
  - **SSN → DeptNum**
  - **SSN → DeptName**
  - **DeptNum → DeptName**
  - **DeptNum → DeptManagerSSN**
- **Is this schema in 3NF? If not, decompose it to 3NF**

3NF - Exercise

- **First check if it is in 1NF?**
  - **As given in the question, birth date and address are not treated as composite attributes in this schema, which means there are no composite or multivalued attributes in the schema**
  - **Thus, it is already in 1NF**
- **Then check if it is in 2NF?**
  - **As there is no combined primary key, so there won't be any partial dependencies possible, hence it is considered in 2NF already**

3NF - Exercise

- **Finally check if it is in 3NF or not?**
  - **For that, we need to find any transitive dependencies from prime attributes to non-prime attributes**
  - **SSN 🎭 EmpName? No transitive dependency here, as there is no other attribute, Z in the schema that will give SSN 🎭 Z and Z 🎭 EmpName**
    - **Then no need to check if determinant is superkey or not, or dependent is prime attribute or not**
  - **SSN 🎭 Bdate? No transitive dependency here, as there is no other attribute, Z in the schema that will give SSN 🎭 Z and Z 🎭 Bdate**
    - **Then no need to check if determinant is superkey or not, or dependent is prime attribute or not**

3NF - Exercise

- **SSN 🎭 Address? No transitive dependency here, as there is no other attribute, Z in the schema that will give SSN 🎭 Z and Z 🎭 Address**
  - **Then no need to check if determinant is superkey or not, or dependent is prime attribute or not**
- **SSN 🎭 DeptNum? No transitive dependency here, as there is no other attribute, Z in the schema that will give SSN 🎭 Z and Z 🎭 EmpName**
  - **Then no need to check if determinant is superkey or not, or dependent is prime attribute or not**

3NF - Exercise

- **SSN 🎭 DeptName? There is transitive dependency here, as there is DeptNum attribute in the schema that will give SSN 🎭 DeptNum and DeptNum 🎭 DeptName**
- **Then check if determinant is superkey or not?**
  - **For SSN 🎭 DeptNum? SSN is a primary key, so this condition holds then not violating the rules of 3NF**
  - **For DeptNum 🎭 DeptName? DeptNum is not a superkey, so check the dependent then. DeptName is not a prime attribute either, so this transitive dependency causes this schema to be NOT in 3NF**

3NF - Exercise

- **Solution:**
  - **The resultant schemas are in 3NF:**
    - **Employee (SSN, EmpName, Bdate, Address, DeptNum)**
    - **Dept (DeptNum, DeptName, DeptManagerSSN)**

THANK YOU